



Техническое задание к курсовому проекту

1. ОБЩИЕ СВЕДЕНИЯ

1.1. Наименование проекта

База знаний «Интеллектуальный помощник по подготовке к техническим собеседованиям» (далее — БЗ TechInterview)

1.2. Назначение

БЗ TechInterview предназначена для использования в качестве информационного ядра интеллектуальной системы, обеспечивающей:

- персонализированную подготовку к техническим собеседованиям в сфере IT;
- диагностику текущего уровня знаний кандидата;
- генерацию вопросов, задач и рекомендаций по устранению пробелов;
- адаптацию под конкретную роль, уровень и компанию;
- объяснение концепций с различной глубиной детализации.

1.3. Целевая аудитория интеллектуальной системы

Сегмент	Описание
Junior-кандидаты	Начинающие разработчики, готовящиеся к первому техническому собеседованию
Middle-кандидаты	Разработчики с 2–5 годами опыта, переходящие в новую компанию или на новый стек
Senior-кандидаты	Опытные инженеры, готовящиеся к собеседованиям в ведущие технологические компании (FAANG-уровень)
Переквалифицирующиеся специалисты	Профессионалы из смежных областей, входящие в IT
Менторы и интервьюеры	Специалисты, проводящие собеседования и нуждающиеся в структурированном банке вопросов

1.4. Команда проекта

Роль	Зона ответственности
Инженер знаний №1	Предметная область «Фундаментальные основы Computer Science»
Инженер знаний №2	Предметная область «Проектирование и архитектура программных систем»
Инженер знаний №3	Предметная область «Процесс собеседования и soft skills»

Координацию и интеграцию выполняют все три участника совместно. Один из участников назначается ответственным за глобальную предметную область и финальную интеграцию.

1.5. Формат результата

Структурированные естественноречевые документы в формате Markdown (.md), организованные в единую файловую структуру с перекрёстными ссылками, метаданными и унифицированной разметкой для последующей автоматизированной обработки интеллектуальной системой.

2. АРХИТЕКТУРА БАЗЫ ЗНАНИЙ

2.1. Уровневая структура

```

text
УРОВЕНЬ 0 – ГЛОБАЛЬНАЯ ПРЕДМЕТНАЯ ОБЛАСТЬ
|   «Подготовка к техническим собеседованиям»
|   (интеграционный уровень)
|
├── УРОВЕНЬ 1 – ПРЕДМЕТНАЯ ОБЛАСТЬ 1
|   «Фундаментальные основы Computer Science»
|
├── УРОВЕНЬ 1 – ПРЕДМЕТНАЯ ОБЛАСТЬ 2
|   «Проектирование и архитектура программных систем»
|
└── УРОВЕНЬ 1 – ПРЕДМЕТНАЯ ОБЛАСТЬ 3
    «Процесс собеседования и soft skills»

```

2.2. Обоснование декомпозиции

Выбранное разделение отражает три фундаментальных измерения, оцениваемых на техническом собеседовании:

1. **Что кандидат знает** — теоретическая база, алгоритмическое мышление, владение фундаментальными концепциями (ПО 1)
2. **Как кандидат проектирует** — умение принимать архитектурные решения, работать с масштабом, декомпозировать сложные системы (ПО 2)
3. **Как кандидат коммуницирует** — способность объяснять, задавать вопросы, работать под давлением, вести диалог с интервьюером (ПО 3)

2.3. Глобальная предметная область (Уровень 0)

Глобальная ПО выполняет функции интеграции и содержит:

2.3.1. Глоссарий общих терминов

Единый словарь терминов, используемых во всех трёх ПО. Включает как технические термины (API, latency, trade-off), так и терминологию рекрутинга (screening, on-site, offer negotiation).

2.3.2. Онтология верхнего уровня

Документ, описывающий:

- ключевые сущности глобальной ПО и связи между ними;
- точки пересечения между тремя ПО;
- иерархию концептов от общего к частному;
- маппинг «тема → этап собеседования → уровень кандидата».

2.3.3. Матрица компетенций

Структурированное описание ожидаемых знаний и навыков для каждого уровня (Junior / Middle / Senior / Staff+) по каждой ПО с индикаторами достижения.

2.3.4. Профили ролей

Описание типовых технических ролей (Backend, Frontend, Fullstack, DevOps/SRE, Data Engineer, Mobile, ML Engineer) с указанием приоритетных тем из каждой ПО и веса каждой темы для конкретной роли.

2.3.5. Карты собеседований компаний

Обобщённые описания процессов собеседования в различных типах компаний (FAANG, стартап, enterprise, аутсорс) с указанием типичных этапов и акцентов.

2.3.6. Реестр перекрёстных ссылок

Таблица связей между документами разных ПО с указанием типа связи.

2.3.7. Учебные траектории

Рекомендуемые последовательности изучения материалов для различных комбинаций «роль × уровень × срок подготовки».

3. СОСТАВ И СОДЕРЖАНИЕ ПРЕДМЕТНЫХ ОБЛАСТЕЙ

3.1. Предметная область 1: «Фундаментальные основы Computer Science» (Инженер знаний №1)

3.1.1. Тематические разделы

№	Раздел	Описание содержания
1.1	Алгоритмическая сложность	Big O нотация, временная и пространственная сложность, амортизированный анализ, классы сложности (P, NP). Типичные вопросы на собеседовании
1.2	Структуры данных	Массивы, связанные списки, стеки, очереди, деревья (BST, AVL, красно-чёрные, B-деревья), кучи, хеш-таблицы, графы, trie, disjoint set. Для каждой: устройство, операции, сложность, типовые задачи
1.3	Алгоритмы сортировки и поиска	Базовые (bubble, insertion, selection), эффективные (merge, quick, heap, radix), бинарный поиск и его вариации. Когда что применять
1.4	Алгоритмические парадигмы	Brute force, divide and conquer, greedy, dynamic programming, backtracking, sliding window, two pointers, BFS/DFS. Паттерны распознавания подхода
1.5	Графовые алгоритмы	Обходы (BFS, DFS), кратчайшие пути (Dijkstra, Bellman-Ford, Floyd-Warshall), MST (Kruskal, Prim), топологическая сортировка, обнаружение циклов, сильная связность
1.6	Операционные системы	Процессы и потоки, планирование, управление памятью (виртуальная память, пейджинг), синхронизация (mutex, semaphore, deadlock), файловые системы, системные вызовы
1.7	Компьютерные сети	Модель OSI/TCP-IP, HTTP/HTTPS, TCP/UDP, DNS, WebSocket, REST vs gRPC, TLS/SSL, CDN, load balancing на сетевом уровне
1.8	Базы данных (фундамент)	Реляционная модель, SQL (JOIN, подзапросы, оконные функции, GROUP BY), нормализация, индексы (B-tree, hash, composite), транзакции (ACID), уровни

№	Раздел	Описание содержания
		изоляции, CAP-теорема, BASE, NoSQL типы (document, key-value, column, graph)
1.9	Основы программирования	ООП (инкапсуляция, наследование, полиморфизм, абстракция), SOLID, функциональное программирование (чистые функции, иммутабельность, функции высшего порядка), типизация, обработка ошибок, generics
1.10	Многопоточность и конкурентность	Потоки, процессы, асинхронность, event loop, корутины, race conditions, deadlocks, lock-free структуры, модель акторов, CSP
1.11	Паттерны решения алгоритмических задач	Систематизация типовых паттернов задач с LeetCode/HackerRank: sliding window, two pointers, fast & slow pointers, merge intervals, cyclic sort, top K elements, modified binary search и т.д.

3.1.2. Типы единиц знаний, специфичных для ПО

Тип	Код	Описание
Карточка концепции	CONCEPT	Теоретическая концепция CS (структура данных, алгоритм, принцип)
Карточка задачи	PROBLEM	Алгоритмическая задача с решением и разбором
Паттерн решения	PATTERN	Типовой подход к классу задач
Карточка вопроса	QUESTION	Вопрос, задаваемый на собеседовании, с эталонным ответом
Шпаргалка	CHEATSHEET	Компактная сводка по теме для быстрого повторения
Типичная ошибка	PITFALL	Распространённая ошибка кандидата с объяснением

3.2. Предметная область 2: «Проектирование и архитектура программных систем» (Инженер знаний №2)

3.2.1. Тематические разделы

№	Раздел	Описание содержания
2.1	Принципы проектирования	SOLID, DRY, KISS, YAGNI, Separation of Concerns, Law of Demeter, Composition over Inheritance. Применение на практике с примерами
2.2	Паттерны проектирования (GoF и не только)	Creational (Singleton, Factory, Builder, Prototype), Structural (Adapter, Decorator, Proxy, Facade, Composite), Behavioral (Observer, Strategy, Command, State, Chain of Responsibility, Iterator). Когда применять, антипаттерны
2.3	System Design: основы	Процесс решения system design задачи. Clarifying questions, capacity estimation (back-of-the-envelope), high-level design, deep dive, bottlenecks, trade-offs
2.4	Компоненты распределённых систем	Load balancers, API gateways, caches (CDN, application cache, distributed cache), message queues (Kafka, RabbitMQ), databases (SQL vs NoSQL, sharding, replication), blob storage, search engines (Elasticsearch)
2.5	Масштабирование	Vertical vs horizontal scaling, stateless services, data partitioning (sharding strategies: hash, range, geographic), replication (master-slave, multi-master), consistent hashing
2.6	Надёжность и отказоустойчивость	Availability vs consistency, redundancy, failover, circuit breaker, retry with backoff, health checks, graceful degradation, SLA/SLO/SLI
2.7	Типовые system design задачи	URL shortener, Twitter/Instagram feed, chat system, notification service, rate limiter, web crawler, video streaming (YouTube), ride sharing (Uber), search autocomplete, distributed file storage и др. Для каждой: постановка, ключевые решения, разбор
2.8	API Design	REST principles, API versioning, pagination, rate limiting, authentication/authorization (OAuth, JWT), GraphQL vs REST vs gRPC, backward compatibility, idempotency
2.9	Микросервисы vs монолит	Архитектурные стили, преимущества и недостатки, service mesh, service discovery, saga pattern, event-driven architecture, CQRS, event sourcing
2.10	Безопасность	OWASP Top 10, SQL injection, XSS, CSRF, authentication vs authorization, encryption at rest / in transit, secrets management, principle of least privilege
2.11	DevOps и инфраструктура (для собеседований)	CI/CD pipeline, контейнеризация (Docker), оркестрация (Kubernetes основы), IaC (Terraform концептуально), мониторинг и observability (logging, metrics, tracing), blue-

№	Раздел	Описание содержания
		green / canary deployments
2.12	Data-intensive системы	Batch vs stream processing, data pipelines, data warehousing vs data lake, ETL/ELT, MapReduce концептуально, Lambda/Kappa архитектуры

3.2.2. Типы единиц знаний, специфичных для ПО

Тип	Код	Описание
Карточка компонента	COMPONENT	Компонент распределённой системы (кеш, очередь, балансировщик)
Карточка паттерна проектирования	DESIGNPAT	Паттерн GoF или архитектурный паттерн
Разбор system design задачи	SYSDESIGN	Полный разбор задачи по проектированию системы
Карточка trade-off	TRADEOFF	Описание компромисса между решениями
Карточка вопроса	QUESTION	Вопрос собеседования с эталонным ответом
Чек-лист проектирования	CHECKLIST	Список аспектов, которые нужно рассмотреть при проектировании
Типичная ошибка	PITFALL	Распространённая ошибка при проектировании

3.3. Предметная область 3: «Процесс собеседования и soft skills» (Инженер знаний №3)

3.3.1. Тематические разделы

№	Раздел	Описание содержания
3.1	Структура технического собеседования	Типичные этапы: screening (HR, recruiter), phone screen (technical), coding interview, system design interview, behavioral interview, team match, hiring committee. Что ожидать на каждом
3.2	Форматы технических интервью	Live coding, take-home assignment, whiteboard, pair programming, code review, presentation. Особенности подготовки к каждому
3.3	Методика решения задач на собеседовании	Фреймворк решения: понять задачу → уточняющие вопросы → примеры → brute force → оптимизация → код → тесты → сложность. «Thinking out loud»
3.4	Коммуникация на техническом собеседовании	Как озвучивать ход мысли, как задавать уточняющие вопросы, как обсуждать trade-offs, как реагировать на подсказки, как признавать незнание, управление временем
3.5	Behavioral-интервью	Метод STAR (Situation, Task, Action, Result), типичные вопросы (leadership, conflict, failure, teamwork, ambiguity), подготовка историй, Amazon Leadership Principles, Google «Googleness»
3.6	Резюме и самопрезентация	Структура технического резюме, описание проектов (STAR для резюме), «Tell me about yourself», elevator pitch, LinkedIn-профиль, портфолио
3.7	Стратегия подготовки	Планирование подготовки (1 месяц, 3 месяца, 6 месяцев), spaced repetition, mock interviews, отслеживание прогресса, выбор задач по сложности, работа над слабыми сторонами
3.8	Психология собеседования	Управление тревожностью, синдром самозванца, работа под давлением, mindset (growth vs fixed), восстановление после отказов, мотивация
3.9	Переговоры и оценка оффера	Структура компенсации (base, bonus, equity, RSU, signing bonus), negotiation tactics, сравнение офферов, вопросы работодателю, red/green flags
3.10	Специфика компаний	FAANG (Google, Meta, Amazon, Apple, Netflix, Microsoft), финтех, стартапы, enterprise, remote-first. Особенности процесса, культурный fit, bar raiser
3.11	Типичные ошибки кандидатов	Систематизация распространённых ошибок на каждом этапе: от подачи заявки до переговоров по офферу
3.12	Этика и профессионализм	Честность на собеседовании, NDA и конфиденциальность, отношение к списыванию, конфликт интересов, профессиональная репутация

3.3.2. Типы единиц знаний, специфичных для ПО

Тип	Код	Описание
Карточка этапа собеседования	STAGE	Описание этапа с рекомендациями по прохождению
Карточка behavioral-вопроса	BEHAVIORAL	Вопрос с рекомендациями по структуре ответа
Стратегия подготовки	STRATEGY	Структурированный план подготовки для конкретной ситуации
Карточка компании/типа компании	COMPANY	Профиль процесса собеседования
Карточка soft skill	SOFTSKILL	Навык с описанием, примерами проявления, способами развития
Карточка ошибки кандидата	PITFALL	Типичная ошибка с анализом и рекомендациями
Скрипт/фреймворк	FRAMEWORK	Готовый фреймворк действий (STAR, метод решения задач и т.д.)
Чек-лист	CHECKLIST	Проверочный список перед собеседованием / этапом

4. ФОРМАТ И ШАБЛОНЫ ДОКУМЕНТОВ

4.1. Общие требования к формату

- **Формат файлов:** Markdown (.md)
- **Кодировка:** UTF-8
- **Язык:** русский (технические термины — на английском с русским пояснением при первом использовании)
- **Стиль изложения:** точный, практико-ориентированный, с примерами кода и конкретными рекомендациями
- **Каждый документ** — одна единица знаний
- **Имена файлов:** латиница, snake_case, с префиксом ПО (например, `cs_concept_hash_table.md`, `arch_sysdesign_url_shortener.md`, `process_behavioral_star_method.md`)
- **Код в примерах:** псевдокод или Python (как lingua franca); при необходимости — с пометкой о языке

4.2. Обязательный блок метаданных

Каждый документ начинается с YAML-фронтматтера:

```
YAML
---
id:"DOMAIN-CATEGORY-NNN"
title:"Название единицы знаний"
domain:"cs | arch | process | global"
category:"concept | problem | pattern | question | sysdesign |
          component | tradeoff | designpat | stage | behavioral |
          strategy | company | softskill | pitfall | framework |
          checklist | cheatsheet | glossary | cross | profile |
          trajectory"
tags:
  - ter1
  - ter2
  - ter3
difficulty:"junior | middle | senior | staff"
roles:
  - "backend"
  - "frontend"
  - "fullstack"
  - "devops"
  - "data_engineer"
  - "mobile"
  - "ml_engineer"
```

```

interview_stage:
  -"phone_screen | coding | system_design | behavioral | take_home"
frequency:"high | medium | low"          # Как часто спрашивают
related:
  -"ID связанного документа 1"
  -"ID связанного документа 2"
prerequisites:
  -"ID документа-пререквизита"
sources:
  -"Ссылка или библиографическое описание"
author:"Имя инженера знаний"
version:"1.0"
date_created:"YYYY-MM-DD"
date_updated:"YYYY-MM-DD"
status:"draft | review | approved"
---
```

4.3. Шаблоны единиц знаний

4.3.1. Шаблон: Карточка концепции CS

```

Markdown
---
[метаданные, category: "concept"]
---

# [Название концепции]

## Определение
[Краткое и точное определение – 1–3 предложения]

## Суть концепции
[Развёрнутое объяснение: что это, зачем нужно,
как работает – 5–15 предложений.
Объяснение строится «от простого к сложному»]

## Ключевые характеристики
- **Характеристика 1**:: значение / описание
- **Характеристика 2**:: значение / описание
- **Временная сложность**:: 0(...)
- **Пространственная сложность**:: 0(...)

## Визуальное представление
[Текстовая ASCII-диаграмма или описание структуры]

## Пример кода
```python
Краткий, чистый пример реализации или использования
```

## Когда используется

- [Сценарий 1]
- [Сценарий 2]

## Сравнение с альтернативами

Критерий	[Эта концепция]	[Альтернатива 1]	[Альтернатива 2]
...	...	...	...

## Типичные вопросы на собеседовании

1. [Вопрос 1] — краткий ответ
2. [Вопрос 2] — краткий ответ
3. [Вопрос 3] — краткий ответ

## Типичные ошибки кандидатов

- [Ошибка 1 — почему неправильно, как правильно]
- [Ошибка 2]

## Уровень по глубине

- **Junior:** [что должен знать]
- **Middle:** [что должен знать дополнительно]
- **Senior:** [что должен знать дополнительно]

## Связанные темы

- [[ID]] — [Название и тип связи]

## Источники

1. [Ссылка]

text

#### 4.3.2. Шаблон: Карточка алгоритмической задачи

```markdown

[метаданные, category: "problem"]

[Название задачи]

Условие

[Формулировка задачи — чёткая, однозначная]

Примеры

Пример 1

- ****Вход****: ...

- ****Выход****: ...

- ****Объяснение****: ...

Пример 2

- ****Вход****: ...

- ****Выход****: ...

```

## Ограничения
- [Ограничение на размер входа]
- [Ограничение на значения]
- [Ограничение по времени/памяти]

## Уточняющие вопросы кандидата
[Какие вопросы следует задать интервьюеру перед решением]
1. [Вопрос 1] — зачем важен
2. [Вопрос 2] — зачем важен

## Подход к решению

### Шаг 1: Brute Force
- Идея: [описание наивного подхода]
- Сложность:  $O(\dots)$  время,  $O(\dots)$  память
- Почему не оптимально: [объяснение]

### Шаг 2: Оптимизация
- Паттерн: [какой паттерн применяется]
- Ключевое наблюдение: [инсайт, ведущий к оптимизации]
- Идея: [описание оптимального подхода]

## Оптимальное решение
```python
def solution(input):
 # Подробно комментированный код
 pass

```

## Анализ сложности

- **Временная**:  $O(\dots)$  — [обоснование]
- **Пространственная**:  $O(\dots)$  — [обоснование]

## Edge Cases

- [Edge case 1]
- [Edge case 2]
- [Edge case 3]

## Как это проверяет интервьюер

[Какие качества кандидата оценивает эта задача]

## Вариации задачи

- [Вариация 1 — как меняется решение]
- [Вариация 2]

## Связанные задачи и концепции

- [[ID]] — [связь]

## Источники

1. [LeetCode / другой ресурс]

text

#### 4.3.3. Шаблон: Разбор System Design задачи

```markdown

[метаданные, category: "sysdesign"]

[Название системы: «Спроектируйте ...»]

Постановка задачи

[Как обычно звучит задача на собеседовании – 2–3 предложения]

Шаг 1: Уточнение требований

Функциональные требования

1. [Требование 1]
2. [Требование 2]
3. [Требование 3]

Нефункциональные требования

1. [Требование: availability, consistency, latency, etc.]
2. [Требование]

Уточняющие вопросы интервьюеру

1. [Вопрос] – [почему важен]
2. [Вопрос] – [почему важен]

Допущения

- [Допущение 1]
- [Допущение 2]

Шаг 2: Capacity Estimation (Back-of-the-envelope)

- **Пользователи**: ... DAU / MAU
- **Запросы**: ... RPS (чтение), ... RPS (запись)
- **Хранилище**: ... TB / год
- **Пропускная способность**: ... Gbps
- **Расчёт**: [пошаговый расчёт]

Шаг 3: High-Level Design

[Текстовое описание архитектуры + ASCII-диаграмма]

[Client] → [Load Balancer] → [API Gateway]

↓

[Service A] [Service B]

↓ ↓

[Database] [Cache] → [Database]

```

text

### Основные компоненты
1. **[Компонент 1]**: назначение, обоснование выбора
2. **[Компонент 2]**: назначение, обоснование выбора

## Шаг 4: Deep Dive

### [Аспект 1: например, Data Model]
[Детальное описание]

### [Аспект 2: например, API Design]
[Детальное описание]

### [Аспект 3: например, Scaling Strategy]
[Детальное описание]

## Шаг 5: Trade-offs и альтернативы
Решение	За	Против	Когда выбирать
[Вариант A]	...	...	...
[Вариант B]	...	...	...

## Шаг 6: Bottlenecks и их решения
1. **[Bottleneck 1]**: [как решать]
2. **[Bottleneck 2]**: [как решать]

## Расширение (если останется время)
- [Дополнительная функциональность 1]
- [Дополнительная функциональность 2]

## Критерии оценки интервьюером
- [Что ожидают от Junior]
- [Что ожидают от Middle]
- [Что ожидают от Senior]

## Типичные ошибки кандидатов
- [Ошибка 1]
- [Ошибка 2]

## Связанные компоненты и концепции
- [[ID]] – [связь]

## Источники
1. [Ссылка]

```

4.3.4. Шаблон: Карточка behavioral-вопроса

```

Markdown
---
[метаданные, category: "behavioral"]
---

```

```

# [Формулировка вопроса]

## Что оценивает
[Какие качества / компетенции проверяет этот вопрос]

## Почему задают
[Контекст: зачем компаниям важен этот аспект – 2–5 предложений]

## Структура ответа (STAR)
### Situation
[Рекомендации: какую ситуацию выбрать, что описать]

### Task
[Рекомендации: как сформулировать задачу/вызов]

### Action
[Рекомендации: на чём делать акцент в описании действий]

### Result
[Рекомендации: как описать результат, метрики]

## Пример хорошего ответа
[Полный пример ответа с комментариями]

## Пример плохого ответа
[Пример с разбором – что не так и почему]

## Вариации вопроса
- [Другая формулировка 1]
- [Другая формулировка 2]

## Red Flags в ответах кандидатов
- [Red flag 1]
- [Red flag 2]

## Компании, которые часто задают
- [Тип компании / конкретная компания + контекст]

## Связанные темы
- [[ID]] – [связь]

```

4.3.5. Шаблон: Карточка типичной ошибки кандидата

```

Markdown
---
[метаданные, category: "pitfall"]
---

# [Название ошибки]

## Описание
[Что именно делает кандидат неправильно – 2–5 предложений]

```

```

## На каком этапе возникает
- [Этап собеседования]
- [Тип задачи / ситуации]

## Почему это проблема
[Как это воспринимается интервьюером – 3–5 предложений]

## Пример ошибки
[Конкретный пример: что сказал/сделал кандидат]

## Как правильно
[Правильное поведение / подход с примером]

## Как избежать
1. [Рекомендация 1]
2. [Рекомендация 2]
3. [Рекомендация 3]

## Уровни кандидатов, подверженные ошибке
- [Junior: почему]
- [Middle: почему]
- [Senior: почему, если применимо]

## Связанные темы
- [[ID]] – [связь]

```

4.3.6. Шаблон: Карточка trade-off

```

Markdown
---
[метаданные, category: "tradeoff"]
---

# [Название trade-off: X vs Y]

## Суть компромисса
[Почему нельзя получить и X, и Y одновременно – 3–7 предложений]

## Вариант А: [Название]
### Преимущества
- [Плюс 1]
- [Плюс 2]

### Недостатки
- [Минус 1]
- [Минус 2]

### Когда выбирать
- [Сценарий 1]
- [Сценарий 2]

## Вариант В: [Название]

```

```

### Преимущества
- [Плюс 1]
- [Плюс 2]

### Недостатки
- [Минус 1]
- [Минус 2]

### Когда выбирать
- [Сценарий 1]
- [Сценарий 2]

## Сравнительная таблица
Критерий	Вариант А	Вариант В
...	...	...

## Реальные примеры
- **[Система/компания 1]**: выбрали [вариант], потому что [причина]
- **[Система/компания 2]**: выбрали [вариант], потому что [причина]

## Как обсуждать на собеседовании
[Рекомендации: как продемонстрировать понимание trade-off]

## Связанные темы
- [[ID]] – [связь]

```

4.3.7. Шаблон: Глоссарная статья

```

Markdown
---
[метаданные, category: "glossary"]
---

# [Термин (English)]

## Определение
[Точное определение – 1–3 предложения]

## Перевод / русскоязычный аналог
[Русский вариант, если устоялся]

## Синонимы
- [Синоним 1]
- [Синоним 2]

## Контекст использования
[В каких ПО, разделах, на каких этапах собеседования
используется]

## Примеры употребления
- "[Пример предложения с термином – как на собеседовании]"
- "[Пример предложения с термином – как в документации]"

```

```
## Связанные термины
- [[ID]] – [термин]
```

4.3.8. Шаблон: Шпаргалка

```
Markdown
---
[метаданные, category: "cheatsheet"]
---

# [Тема]: Шпаргалка

## Ключевые факты
Пункт	Значение
...	...

## Формулы / правила
- [Формула 1]
- [Правило 1]

## Частые вопросы и быстрые ответы
Вопрос	Ответ (1–2 предложения)
...	...

## Мнемоники
- [Мнемоника для запоминания]

## Что повторить за 5 минут до собеседования
1. [Пункт 1]
2. [Пункт 2]
3. [Пункт 3]
```

5. ФАЙЛОВАЯ СТРУКТУРА ПРОЕКТА

```
text
knowledge_base_techinterview/
|
|— README.md
|— CONTRIBUTING.md
|— CHANGELOG.md
|
|— global/
|   |— glossary/
|   |   |— global_glossary_api.md
|   |   |— global_glossary_latency.md
|   |   |— global_glossary_tradeoff.md
|   |   └ ...
|   |— ontology/
|   |   └ global_ontology_main.md
```

```

├── competency_matrix/
│   ├── global_matrix_junior.md
│   ├── global_matrix_middle.md
│   ├── global_matrix_senior.md
│   └── global_matrix_staff.md
├── role_profiles/
│   ├── global_profile_backend.md
│   ├── global_profile_frontend.md
│   ├── global_profile_fullstack.md
│   ├── global_profile_devops.md
│   ├── global_profile_data_engineer.md
│   ├── global_profile_mobile.md
│   └── global_profile_ml_engineer.md
├── company_maps/
│   ├── global_company_faang.md
│   ├── global_company_startup.md
│   ├── global_company_enterprise.md
│   └── global_company_outsource.md
├── trajectories/
│   ├── global_trajectory_junior_backend_1month.md
│   ├── global_trajectory_middle_fullstack_3months.md
│   ├── global_trajectory_senior_faang_6months.md
│   └── ...
├── cross_domain/
│   ├── global_cross_cs_and_sysdesign.md
│   ├── global_cross_coding_and_communication.md
│   ├── global_cross_architecture_and_behavioral.md
│   └── ...
└── index.md

├── cs/                                     # ПО «Фундаментальные основы CS»
│   ├── glossary/
│   ├── concepts/
│   │   ├── data_structures/
│   │   │   ├── cs_concept_array.md
│   │   │   ├── cs_concept_linked_list.md
│   │   │   ├── cs_concept_hash_table.md
│   │   │   ├── cs_concept_binary_tree.md
│   │   │   ├── cs_concept_heap.md
│   │   │   ├── cs_concept_graph.md
│   │   │   ├── cs_concept_trie.md
│   │   │   └── ...
│   │   ├── algorithms/
│   │   │   ├── cs_concept_binary_search.md
│   │   │   ├── cs_concept_quicksort.md
│   │   │   ├── cs_concept_dynamic_programming.md
│   │   │   ├── cs_concept_bfs.md
│   │   │   ├── cs_concept_dijkstra.md
│   │   │   └── ...
│   │   ├── os/
│   │   ├── networking/
│   │   ├── databases/
│   │   ├── programming/
│   │   └── concurrency/

```

```

├── problems/
│   ├── arrays/
│   ├── strings/
│   ├── trees/
│   ├── graphs/
│   ├── dynamic_programming/
│   ├── sliding_window/
│   └── ...
├── patterns/
│   ├── cs_pattern_sliding_window.md
│   ├── cs_pattern_two_pointers.md
│   ├── cs_pattern_fast_slow_pointers.md
│   ├── cs_pattern_merge_intervals.md
│   ├── cs_pattern_top_k_elements.md
│   └── ...
├── questions/
├── cheatsheets/
├── pitfalls/
└── index.md

├── arch/                                     # ПО «Проектирование и архитектура»
│   ├── glossary/
│   ├── components/
│   │   ├── arch_component_load_balancer.md
│   │   ├── arch_component_cache.md
│   │   ├── arch_component_message_queue.md
│   │   ├── arch_component_cdn.md
│   │   ├── arch_component_api_gateway.md
│   │   └── ...
│   ├── design_patterns/
│   │   ├── creational/
│   │   ├── structural/
│   │   └── behavioral/
│   ├── system_design/
│   │   ├── arch_sysdesign_url_shortener.md
│   │   ├── arch_sysdesign_twitter_feed.md
│   │   ├── arch_sysdesign_chat_system.md
│   │   ├── arch_sysdesign_rate_limiter.md
│   │   ├── arch_sysdesign_notification_service.md
│   │   ├── arch_sysdesign_video_streaming.md
│   │   └── ...
│   ├── tradeoffs/
│   │   ├── arch_tradeoff_sql_vs_nosql.md
│   │   ├── arch_tradeoff_consistency_vs_availability.md
│   │   ├── arch_tradeoff_monolith_vs_microservices.md
│   │   └── ...
│   ├── questions/
│   ├── checklists/
│   ├── pitfalls/
│   └── index.md

├── process/                                # ПО «Процесс собеседования и soft skills»
│   ├── glossary/
│   └── stages/

```

```

├── process_stage_hr_screening.md
├── process_stage_phone_screen.md
├── process_stage_coding_interview.md
├── process_stage_system_design_interview.md
├── process_stage_behavioral_interview.md
├── process_stage_team_match.md
├── process_stage_offer_negotiation.md
├── behavioral/
│   ├── process_behavioral_tell_about_yourself.md
│   ├── process_behavioral_conflict_resolution.md
│   ├── process_behavioral_biggest_failure.md
│   ├── process_behavioral_leadership.md
│   ├── process_behavioral_ambiguity.md
│   └── ...
├── frameworks/
│   ├── process_framework_star.md
│   ├── process_framework_problem_solving.md
│   ├── process_framework_sysdesign_approach.md
│   └── ...
├── strategies/
│   ├── process_strategy_1month_plan.md
│   ├── process_strategy_3month_plan.md
│   ├── process_strategy_mock_interviews.md
│   └── ...
├── softskills/
├── companies/
├── psychology/
├── negotiation/
├── pitfalls/
├── checklists/
│   ├── process_checklist_before_interview.md
│   ├── process_checklist_after_interview.md
│   └── ...
├── index.md
└── _meta/
    ├── id_registry.md
    ├── cross_reference_table.md
    ├── tag_taxonomy.md
    ├── sources_master_list.md
    ├── validation_checklist.md
    └── difficulty_calibration.md # Калибровка уровней сложности

```

6. ТРЕБОВАНИЯ К СОДЕРЖАНИЮ

6.1. Требования к достоверности

1. Каждая единица знаний должна быть подтверждена **минимум одним авторитетным источником**
2. Приоритетные источники:
 - Фундаментальные учебники (CLRS «Introduction to Algorithms», Tanenbaum «Computer Networks», «Designing Data-Intensive Applications» М. Kleppmann и т.д.)
 - Официальная документация технологий

- Публикации инженеров крупных компаний (Engineering blogs: Google, Meta, Netflix, Uber, Airbnb)
- Рецензируемые ресурсы (System Design Primer, Grokking the System Design Interview, NeetCode)
- Подтверждённый практический опыт (опубликованные разборы процессов собеседования)

3. Для алгоритмических задач — обязательна **верификация решения** (корректность, сложность)

4. Для behavioral-вопросов — указание источника (конкретная компания, Leadership Principles и т.д.)

6.2. Требования к актуальности

1. Информация о процессах собеседования в компаниях — не старше **2 лет** (с указанием даты актуализации)

2. Технологические концепции должны отражать **текущее состояние индустрии** (не устаревшие подходы)

3. При описании технологий указывать **тренды**: что набирает популярность, что уходит

4. Каждый документ имеет поле `date_updated` для отслеживания свежести

6.3. Требования к практичности

1. **Каждая теоретическая концепция** должна содержать:

- пример, как она спрашивается на собеседовании;
- рекомендацию, на каком уровне и для каких ролей она критична.

2. **Каждая задача** должна содержать:

- ход решения «как бы решал кандидат на собеседовании» (а не просто код);
- уточняющие вопросы, которые нужно задать.

3. **Каждый behavioral-вопрос** должен содержать:

- пример хорошего и плохого ответа;
- фреймворк для структурирования собственного ответа.

6.4. Требования к калибровке сложности

Уровни сложности (`difficulty`) калибруются единообразно:

| Уровень | Для CS (coding) | Для Architecture | Для Process |
|---------------|--|---|--|
| junior | LeetCode Easy, базовые структуры данных | Понимание клиент-сервер, основы REST | Умение рассказать о себе, базовая коммуникация |
| middle | LeetCode Medium, DP основы, графы | Проектирование простых систем (URL shortener, rate limiter) | STAR-метод, осознанная коммуникация trade-offs |
| senior | LeetCode Hard, сложные паттерны, оптимизации | Проектирование масштабных систем (Twitter, YouTube) | Лидерство, стратегическое мышление, менторинг |
| staff | Нестандартные задачи, дизайн алгоритмов | Кросс-системная архитектура, организационные решения | Influence without authority, org-level impact |

Калибровка фиксируется в `_meta/difficulty_calibration.md`.

6.5. Требования к полноте

Минимальный объём по каждой ПО:

| Тип единицы знаний | CS | Архитектура | Процесс |
|---|------|-------------|---------|
| Глоссарные статьи | ≥ 40 | ≥ 35 | ≥ 25 |
| Карточки концепций / компонентов / этапов | ≥ 50 | ≥ 30 | ≥ 20 |
| Задачи / разборы system design / behavioral-вопросы | ≥ 40 | ≥ 15 | ≥ 25 |
| Паттерны / паттерны проектирования / фреймворки | ≥ 15 | ≥ 25 | ≥ 10 |

| Тип единицы знаний | CS | Архитектура | Процесс |
|----------------------------------|--------------|--------------|--------------|
| Вопросы собеседования с ответами | ≥ 30 | ≥ 20 | ≥ 15 |
| Шпаргалки | ≥ 10 | ≥ 8 | ≥ 5 |
| Типичные ошибки | ≥ 15 | ≥ 10 | ≥ 15 |
| Trade-offs | — | ≥ 12 | — |
| Чек-листы | — | ≥ 5 | ≥ 8 |
| Стратегии подготовки | — | — | ≥ 8 |
| Итого по ПО | ≥ 200 | ≥ 160 | ≥ 131 |

Глобальная ПО: ≥ 7 профилей ролей + ≥ 4 матрицы компетенций + ≥ 4 карты компаний + ≥ 6 учебных траекторий + ≥ 10 межпредметных документов + ≥ 30 общих глоссарных статей + онтология = **≥ 62 документа**.

Общий минимальный объём БЗ: ≥ 553 единицы знаний.

6.6. Требования к непротиворечивости

- Одни и те же концепции в разных ПО описываются **согласованно**
- Уровни сложности **калиброваны** единообразно (см. п. 6.4)
- Терминология — строго по глоссарию
- Рекомендации по подготовке **не должны противоречить** друг другу между ПО
- При наличии спорных мнений (например, «нужно ли учить алгоритмы для реальной работы») — фиксировать **все позиции** с аргументами

7. СИСТЕМА ИДЕНТИФИКАЦИИ И СВЯЗЕЙ

7.1. Формат идентификаторов

```
text
[ДОМЕН] - [КАТЕГОРИЯ] - [НОМЕР]
```

Примеры:

- CS-CONCEPT-042** — концепция №042 из ПО Computer Science
- CS-PROBLEM-015** — задача №015 из ПО Computer Science
- ARCH-SYSDSIGN-003** — system design задача №003 из ПО Архитектура
- ARCH-TRADEOFF-007** — trade-off №007 из ПО Архитектура
- PROC-BEHAVIORAL-012** — behavioral-вопрос №012 из ПО Процесс
- GLB-PROFILE-002** — профиль роли №002 из глобальной ПО
- GLB-TRAJECTORY-004** — учебная траектория №004

7.2. Коды доменов

| Домен | Код |
|-----------------------|------|
| Глобальная ПО | GLB |
| Computer Science | CS |
| Архитектура | ARCH |
| Процесс собеседования | PROC |

7.3. Коды категорий

| Категория | Код | ПО |
|------------|------------------------|------------|
| GLOSS | Глоссарная статья | Все |
| CONCEPT | Концепция CS | CS |
| PROBLEM | Алгоритмическая задача | CS |
| PATTERN | Паттерн решения задач | CS |
| COMPONENT | Компонент системы | ARCH |
| DESIGNPAT | Паттерн проектирования | ARCH |
| SYSDESIGN | Разбор system design | ARCH |
| TRADEOFF | Компромисс | ARCH |
| STAGE | Этап собеседования | PROC |
| BEHAVIORAL | Behavioral-вопрос | PROC |
| STRATEGY | Стратегия подготовки | PROC |
| COMPANY | Профиль компании/типа | PROC |
| SOFTSKILL | Soft skill | PROC |
| FRAMEWORK | Фреймворк/методика | PROC |
| QUESTION | Вопрос собеседования | Все |
| CHEATSHEET | Шпаргалка | Все |
| PITFALL | Типичная ошибка | Все |
| CHECKLIST | Чек-лист | ARCH, PROC |
| PROFILE | Профиль роли | GLB |
| MATRIX | Матрица компетенций | GLB |
| TRAJECTORY | Учебная траектория | GLB |
| CROSS | Межпредметный документ | GLB |
| ONTO | Онтология | GLB |

7.4. Типы связей между документами

| Код связи | Описание | Пример |
|-------------------------------|--------------------------|--------------------------------------|
| <code>is_part_of</code> | Является частью | Задача → Паттерн |
| <code>relates_to</code> | Тематически связан | Hash Table → Hash Function |
| <code>prerequisite_for</code> | Является предпосылкой | Array → Dynamic Programming |
| <code>depends_on</code> | Зависит от | Dijkstra → Priority Queue |
| <code>alternative_to</code> | Альтернативный подход | BFS → DFS для той же задачи |
| <code>extends</code> | Расширяет / углубляет | BST → AVL Tree |
| <code>applied_in</code> | Применяется в | Consistent Hashing → URL Shortener |
| <code>evaluates</code> | Оценивает (этап → навык) | Coding Interview → Алгоритмы |
| <code>contrasts_with</code> | Противопоставляется | SQL vs NoSQL |
| <code>exemplifies</code> | Является примером | Two Sum → Two Pointers Pattern |
| <code>complements</code> | Дополняет | Coding Skills → Communication Skills |
| <code>warns_about</code> | Предупреждает о рисках | Ошибка → Концепция |

8. ТАКСОНОМИЯ ТЕГОВ

8.1. Обязательные группы тегов

Группа 1 — Техническая тема:

arrays, strings, linked_lists, trees, graphs, hash_tables, heaps, stacks_queues, dynamic_programming, greedy, sorting, searching, re

Группа 2 — Уровень:

junior, middle, senior, staff

Группа 3 — Роль:

backend, frontend, fullstack, devops, data_engineer, mobile, ml_engineer, all_roles

Группа 4 — Этап собеседования:

hr_screening, phone_screen, coding, system_design, behavioral, take_home, team_match, offer

Группа 5 — Тип активности:

теория, практика, решение_задач, проектирование, коммуникация, подготовка, рефлексия

Группа 6 — Частота на собеседованиях:

must_know, frequently_asked, occasionally_asked, rare_but_important

Группа 7 — Приоритет:

core, advanced, specialized, nice_to_have

Полная таксономия фиксируется в `_meta/tag_taxonomy.md`.

13. ПРИЛОЖЕНИЯ

Приложение А. Пример: Карточка концепции CS

```
Markdown
---
id:"CS-CONCEPT-017"
title:"Хеш-таблица (Hash Table)"
domain:"cs"
category:"concept"
tags:
  -hash_tables
  -data_structures
  -must_know
  -core
  -теория
difficulty:"junior"
roles:
  -"all_roles"
interview_stage:
  -"phone_screen"
  -"coding"
frequency:"high"
related:
  -"CS-CONCEPT-018"    # Хеш-функция
  -"CS-CONCEPT-019"    # Коллизии и их разрешение
  -"CS-CONCEPT-003"    # Массив
  -"CS-CONCEPT-042"    # Связный список
  -"CS-PROBLEM-008"      # Two Sum
  -"ARCH-COMPONENT-005"  # Distributed Cache
prerequisites:
  -"CS-CONCEPT-003"    # Массив
sources:
  -"Cormen T.H. et al. Introduction to Algorithms (CLRS),
```

```

    4th Edition, Chapter 11: Hash Tables"
    -"Skiena S.S. The Algorithm Design Manual, 3rd Edition,
      Section 3.7"
author:"Инженер знаний №1"
version:"1.0"
date_created:"2025-03-01"
date_updated:"2025-03-01"
status:"approved"
---
```

Хеш-таблица (Hash Table)

Определение

Хеш-таблица – структура данных, реализующая ассоциативный массив (отображение ключ → значение) на основе хеш-функции, обеспечивающая в среднем $O(1)$ для операций вставки, поиска и удаления.

Суть концепции

Хеш-таблица использует хеш-функцию для преобразования ключа в индекс массива (bucket), где хранится соответствующее значение. Это позволяет получить прямой доступ к данным без перебора, в отличие от массива (поиск по значению $O(n)$) или дерева ($O(\log n)$).

Основная проблема – коллизии: ситуации, когда разные ключи отображаются на один и тот же индекс. Существует два основных способа разрешения коллизий:

1. **Chaining (цепочки)** – каждый bucket содержит связный список элементов с одинаковым хешем.
2. **Open addressing (открытая адресация)** – при коллизии ищется следующий свободный slot (linear probing, quadratic probing, double hashing).

Производительность зависит от качества хеш-функции и load factor (отношение количества элементов к размеру таблицы). При load factor > 0.75 обычно выполняется rehashing – увеличение таблицы с пересчётом хешей.

Ключевые характеристики

- **Поиск (average)**: $O(1)$
- **Поиск (worst case)**: $O(n)$ – при вырождении в список
- **Вставка (average)**: $O(1)$
- **Удаление (average)**: $O(1)$
- **Пространственная сложность**: $O(n)$
- **Упорядоченность**: не гарантируется
- **Допускает null-ключи**: зависит от реализации

Визуальное представление

```

```text
Ключ "apple" → hash("apple") = 3
Ключ "banana" → hash("banana") = 7
Ключ "cherry" → hash("cherry") = 3 ← коллизия!

```

Массив buckets:

- [0] → null
- [1] → null
- [2] → null
- [3] → ("apple", 5) → ("cherry", 8) ← chaining
- [4] → null
- [5] → null
- [6] → null
- [7] → ("banana", 3)

## Пример кода

```
Python
class SimpleHashTable:
 def __init__(self, size=16):
 self.size= size
 self.buckets= [[]for _in range(size)]
 self.count= 0

 def _hash(self, key:str) ->int:
 return hash(key)% self.size

 def put(self, key:str, value) ->None:
 idx= self._hash(key)
 for i, (k, v)in enumerate(self.buckets[idx]):
 if k== key:
 self.buckets[idx][i]= (key, value)
 return
 self.buckets[idx].append((key, value))
 self.count+= 1

 def get(self, key:str):
 idx= self._hash(key)
 for k, vin self.buckets[idx]:
 if k== key:
 return v
 raise KeyError(key)

 def delete(self, key:str) ->None:
 idx= self._hash(key)
 for i, (k, v)in enumerate(self.buckets[idx]):
 if k== key:
 self.buckets[idx].pop(i)
 self.count-= 1
 return
 raise KeyError(key)
```

## Когда используется

- Быстрый поиск / проверка наличия элемента (membership test)
- Подсчёт частоты элементов (frequency counting)
- Группировка данных (grouping by key)

- Кеширование (memoization, LRU cache)
- Устранение дубликатов
- Реализация множеств (set)

## Сравнение с альтернативами

Критерий	Hash Table	BST (сбалансир.)	Sorted Array
Поиск	$O(1)$ avg	$O(\log n)$	$O(\log n)$ binary
Вставка	$O(1)$ avg	$O(\log n)$	$O(n)$ сдвиг
Удаление	$O(1)$ avg	$O(\log n)$	$O(n)$ сдвиг
Порядок	Нет	Да	Да
Min/Max	$O(n)$	$O(\log n)$	$O(1)$
Range query	$O(n)$	$O(\log n + k)$	$O(\log n + k)$

## Типичные вопросы на собеседовании

1. **Как работает hash table внутри?** — описать хеш-функцию, buckets, коллизии, rehashing
2. **Чем HashMap отличается от HashSet?** — Map хранит пары key-value, Set — только ключи (значения boolean)
3. **Когда  $O(1)$  превращается в  $O(n)$ ?** — при множественных коллизиях (плохая хеш-функция, атака)
4. **Когда лучше использовать TreeMap вместо HashMap?** — когда нужен порядок, range queries, min/max

## Типичные ошибки кандидатов

- **Забывают о worst case:** утверждают, что hash table «всегда  $O(1)$ » — это только average case
- **Не упоминают rehashing:** не объясняют, что происходит при заполнении таблицы
- **Путают hash table и hash set:** используют термины взаимозаменяемо
- **Не рассматривают альтернативы:** используют hash table там, где нужен порядок (лучше BST или sorted structure)

## Уровень по глубине

- **Junior:** знать определение, average-case сложности, уметь использовать dict/HashMap для решения задач
- **Middle:** понимать устройство (chaining vs open addressing), load factor, rehashing, уметь объяснить worst case
- **Senior:** знать конкурентные хеш-таблицы (ConcurrentHashMap), consistent hashing для распределённых систем, выбирать между hash table и альтернативами с обоснованием

## Связанные темы

- [[CS-CONCEPT-018]] — Хеш-функция (depends\_on)
- [[CS-CONCEPT-019]] — Коллизии и их разрешение (extends)
- [[CS-CONCEPT-003]] — Массив (prerequisite\_for)
- [[CS-CONCEPT-042]] — Связный список (relates\_to, chaining)
- [[CS-PROBLEM-008]] — Two Sum (applied\_in)
- [[CS-PATTERN-003]] — Frequency counting (applied\_in)
- [[ARCH-COMPONENT-005]] — Distributed Cache (relates\_to)
- [[ARCH-TRADEOFF-008]] — Consistent Hashing vs Range Partitioning (extends)

## Источники

1. Cormen T.H. et al. Introduction to Algorithms (CLRS), 4th Edition, Chapter 11: Hash Tables. MIT Press, 2022.
2. Skiena S.S. The Algorithm Design Manual, 3rd Edition, Section 3.7. Springer, 2020.
3. Python Documentation. Built-in Types: dict. <https://docs.python.org/3/library/stdtypes.html#dict>

text

### Приложение Б. Пример: Разбор system design задачи

```markdown

id: "ARCH-SYSDESIGN-001"

title: "Проектирование URL Shortener (TinyURL)"

domain: "arch"

category: "sysdesign"

tags:

- databases
- caching
- load_balancing
- api_design
- data_modeling
- must_know
- core
- проектирование

difficulty: "middle"

roles:

- "backend"
- "fullstack"

interview_stage:

- "system_design"

frequency: "high"

related:

- "ARCH-COMPONENT-005" # Cache
- "ARCH-COMPONENT-001" # Load Balancer
- "ARCH-COMPONENT-008" # NoSQL Database
- "ARCH-TRADEOFF-001" # SQL vs NoSQL
- "CS-CONCEPT-017" # Hash Table
- "CS-CONCEPT-018" # Hash Function
- "ARCH-DESIGNPAT-012" # Base62 Encoding
- "PROC-FRAMEWORK-003" # Фреймворк решения SD задачи

prerequisites:

- "CS-CONCEPT-017" # Hash Table
- "ARCH-COMPONENT-005" # Cache
- "ARCH-COMPONENT-001" # Load Balancer

sources:

- "Xu A. System Design Interview – An Insider's Guide, Chapter 8: Design a URL Shortener. 2020."
- "The System Design Primer by Donne Martin.
<https://github.com/donnemartin/system-design-primer>"

author: "Инженер знаний №2"

version: "1.0"

date_created: "2025-03-05"

date_updated: "2025-03-05"

status: "approved"

Проектирование URL Shortener (TinyURL)

Постановка задачи

Спроектируйте сервис для сокращения URL-адресов (аналог bit.ly / TinyURL). Пользователь вводит длинный URL, получает короткий. При переходе по короткому URL происходит редирект на оригинальный.

Шаг 1: Уточнение требований

Функциональные требования

1. По длинному URL генерировать уникальный короткий URL
2. По короткому URL перенаправлять на оригинальный
3. Опционально: пользовательский alias, TTL (время жизни), аналитика кликов

Нефункциональные требования

1. Высокая доступность (availability > 99.9%)
2. Низкая задержка редиректа (< 100ms p99)
3. Короткие URL не должны быть предсказуемыми
4. Масштабируемость до миллиардов URL

Уточняющие вопросы интервьюеру

1. Каков ожидаемый объем трафика? (→ capacity estimation)
2. Какова длина короткого URL? (→ пространство ключей)
3. Нужна ли аналитика? (→ дополнительные компоненты)
4. Нужна ли авторизация? (→ сложность API)
5. Какой TTL по умолчанию? (→ стратегия очистки)

Допущения

- 100M новых URL / месяц
- Соотношение чтение:запись = 100:1
- Хранение 5 лет
- Средняя длина оригинального URL: 200 байт

Шаг 2: Capacity Estimation

```text

Запись:

$$100M / \text{месяц} = 100M / (30 \times 24 \times 3600) \approx 40 \text{ URL/сек}$$

Чтение:

$$40 \times 100 = 4000 \text{ redirects/сек}$$

Хранилище (5 лет):

$$100M \times 12 \times 5 = 6B \text{ записей}$$

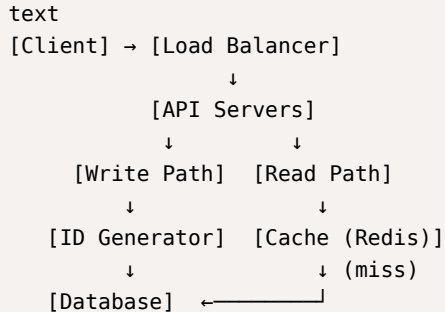
$$6B \times (200 \text{ байт URL} + 8 \text{ байт short} + 8 \text{ байт meta})$$

$$\approx 6B \times 216 \text{ байт} \approx 1.3 \text{ TB}$$

Пространство ключей:

Short URL из 7 символов Base62:  
 $62^7 \approx 3.5$  триллионов (достаточно для 6B)

## Шаг 3: High-Level Design



### Основные компоненты

1. **API Servers (stateless)**: обработка запросов на создание и редирект
2. **ID Generator**: генерация уникальных коротких ключей
3. **Database**: хранение маппинга short → long URL
4. **Cache (Redis)**: кеш для hot URLs (80/20 правило)
5. **Load Balancer**: распределение нагрузки

## Шаг 4: Deep Dive

### 4.1 Генерация короткого URL

#### Подход 1: Hash + обрезка

- Применить MD5/SHA-256 к long URL, взять первые 7 символов
- Плюс: детерминированный (один URL → один short)
- Минус: коллизии, нужна проверка уникальности

#### Подход 2: Counter-based (Auto-increment ID → Base62)

- Глобальный счётчик, конвертация в Base62
- Плюс: гарантированная уникальность, нет коллизий
- Минус: предсказуемость, single point of failure

#### Подход 3: Pre-generated keys (Key Generation Service)

- Заранее сгенерировать пул уникальных ключей в отдельной БД
- Worker берёт ключ из пула при создании URL
- Плюс: быстро, нет коллизий, не предсказуемо
- Минус: дополнительная инфраструктура

**Выбор:** KGS (Key Generation Service) — лучший баланс для масштабируемой системы.

### 4.2 API Design

```

text
POST /api/v1/urls
 Body: { "long_url": "https://...", "ttl": 3600 }
 Response: { "short_url": "https://tinyurl.com/abc1234" }

GET /{short_key}
 Response: 301 Redirect → long_url
 (301 vs 302: 301 кешируется браузером, 302 – нет;
 если нужна аналитика – используем 302)

```

### 4.3 Data Model

```

text
Table: urls
- id: BIGINT (PK)
- short_key: VARCHAR(7) (UNIQUE INDEX)
- long_url: VARCHAR(2048)
- created_at: TIMESTAMP
- expires_at: TIMESTAMP
- user_id: BIGINT (nullable, FK)

```

### 4.4 Caching Strategy

- Cache hot URLs в Redis (LRU eviction)
- Read path: Cache → DB
- Cache hit ratio ожидается ~80% (закон Парето)
- TTL в кеше: совпадает с TTL URL или 24h

## Шаг 5: Trade-offs и альтернативы

| Решение          | За                             | Против               | Когда         |
|------------------|--------------------------------|----------------------|---------------|
| SQL (PostgreSQL) | ACID, надёжность               | Сложнее шардить      | < 1B записей  |
| NoSQL (DynamoDB) | Горизонтальное масштабирование | Eventual consistency | > 1B записей  |
| 301 Redirect     | Меньше нагрузки                | Нет аналитики        | Без аналитики |
| 302 Redirect     | Можно считать клики            | Больше запросов      | С аналитикой  |

## Шаг 6: Bottlenecks и их решения

1. **ID Generator — single point of failure:** → KGS с pre-generated keys, два пула (used/unused)
2. **Database — нагрузка на запись:** → Шардинг по short\_key (range или hash)
3. **Hot URLs:** → Multi-tier caching (CDN → Redis → DB)
4. **Expired URLs:** → Lazy cleanup + periodic background job

## Расширение

- Аналитика кликов (Kafka → Analytics DB)
- Rate limiting (защита от спама)
- Custom aliases
- QR-code генерация

## Критерии оценки интервьюером

- **Middle:** чёткие требования, capacity estimation, рабочий high-level design, один deep dive
- **Senior:** все вышеперечисленное + trade-offs, масштабирование, отказоустойчивость, несколько deep dive с обоснованием выбора

## Типичные ошибки кандидатов

- Пропускают capacity estimation и сразу рисуют архитектуру
- Не обсуждают коллизии при генерации ключей
- Забывают о кешировании (read-heavy система!)
- Не уточняют 301 vs 302
- Не продумывают TTL и очистку expired URLs

## Связанные компоненты и концепции

- [[ARCH-COMPONENT-005]] — Cache / Redis (applied\_in)
- [[ARCH-COMPONENT-001]] — Load Balancer (applied\_in)
- [[ARCH-TRADEOFF-001]] — SQL vs NoSQL (applied\_in)
- [[CS-CONCEPT-017]] — Hash Table (relates\_to)
- [[CS-CONCEPT-018]] — Hash Function (relates\_to)
- [[PROC-FRAMEWORK-003]] — Фреймворк решения SD задачи (is\_part\_of)

## Источники

1. Xu A. System Design Interview – An Insider's Guide, Chapter 8. ByteByteGo, 2020.
2. The System Design Primer — URL Shortener. <https://github.com/donnemartin/system-design-primer>
3. How We Built Bitly — Bitly Engineering Blog.

text

### Приложение В. Пример: Межпредметный документ

```markdown

id: "GLB-CROSS-002"

title: "Связь алгоритмического мышления и System Design"

domain: "global"

category: "cross"

tags:

- algorithms
- system_design
- проектирование
- решение_задач
- advanced

difficulty: "middle"

roles:

- "all_roles"

interview_stage:

- "coding"

```

- "system_design"
frequency: "high"
related:
- "CS-CONCEPT-001"      # Big O нотация
- "CS-PATTERN-001"        # Паттерны решения задач
- "ARCH-SYSDESIGN-001"    # URL Shortener
- "ARCH-TRADEOFF-001"     # SQL vs NoSQL
- "PROC-FRAMEWORK-003"    # Фреймворк решения SD задачи
sources:
- "Xu A. System Design Interview, Chapter 2:
  Back-of-the-envelope Estimation. 2020."
- "McDowell G.L. Cracking the Coding Interview,
  Chapter VII: Technical Questions. 2015."
author: "Инженер знаний №1 + Инженер знаний №2"
version: "1.0"
date_created: "2025-03-10"
date_updated: "2025-03-10"
status: "approved"
---
```

Связь алгоритмического мышления и System Design

Суть связи

Алгоритмическое мышление (ПО 1) и проектирование систем (ПО 2) — не изолированные навыки. Кандидаты, которые рассматривают их отдельно, упускают значительные синергии. Понимание алгоритмов влияет на качество архитектурных решений, а опыт System Design помогает оценивать алгоритмы в контексте реальных ограничений.

Точки пересечения

1. Оценка масштаба (Capacity Estimation ↔ Big O)

Навык оценки сложности алгоритма (Big O) напрямую переносится на back-of-the-envelope calculations в System Design. Тот же навык: «сколько операций / запросов / байт — и укладываемся ли мы в ограничения?»

****Пример**:** оценивая QPS для URL shortener, кандидат применяет тот же навык, что и при оценке сложности вложенных циклов.

2. Структуры данных → Компоненты систем

Многие компоненты распределённых систем являются масштабированными версиями классических структур данных:

| Структура данных | Компонент системы |
|-----------------------|--------------------------------------|
| Hash Table | Distributed Cache (Redis, Memcached) |
| Heap / Priority Queue | Task Scheduler, Rate Limiter |
| Trie | Search Autocomplete |
| Graph | Social Network Graph DB |
| Bloom Filter | Distributed membership test |
| B-Tree / LSM-Tree | Database storage engine |

| Queue / Stack | Message Queue (Kafka, RabbitMQ) |

3. Паттерны решения задач → Архитектурные паттерны

| Алгоритмический паттерн | Архитектурный аналог |
|-----|-----|
| Divide and Conquer | Sharding, MapReduce |
| Caching (Memoization) | CDN, Application Cache |
| Two-phase approach | Two-Phase Commit |
| Event-driven (Observer) | Event-Driven Architecture |
| Pipeline | Data Pipeline, CI/CD |

4. Trade-off мышление

И в алгоритмах (time vs space), и в System Design (consistency vs availability) ключевой навык – осознанный выбор компромисса с обоснованием.

Рекомендации для подготовки

1. При изучении структур данных – задавать вопрос «где это используется в реальных системах?»
2. При проектировании систем – задавать вопрос «какая структура данных лежит в основе этого компонента?»
3. Тренировать capacity estimation как отдельный навык
4. Решать задачи, находящиеся на стыке: rate limiter, LRU cache, consistent hashing

Как проявляется на собеседовании

- На coding interview могут спросить: «Как бы вы масштабировали это решение на 1B записей?»
- На system design могут попросить: «Покажите псевдокод ключевого алгоритма»
- Интервьюеры ценят, когда кандидат демонстрирует связь между уровнями абстракции

Связанные темы

- [[CS-CONCEPT-001]] – Big O нотация (relates_to)
- [[CS-PATTERN-001]] – Паттерны решения задач (relates_to)
- [[ARCH-SYSDSIGN-001]] – URL Shortener (exemplifies)
- [[ARCH-TRADEOFF-001]] – SQL vs NoSQL (exemplifies)
- [[PROC-FRAMEWORK-003]] – Фреймворк решения SD (complements)
- [[GLB-CROSS-001]] – Связь coding и communication (complements)

Источники

1. Xu A. System Design Interview, Chapter 2. 2020.
2. McDowell G.L. Cracking the Coding Interview, Chapter VII. 2015.
3. Kleppmann M. Designing Data-Intensive Applications, Chapter 1: Foundations. O'Reilly, 2017.

Приложение Г. Пример: Карточка behavioral-вопроса

```

Markdown
---
id:"PROC-BEHAVIORAL-003"
title:"Tell me about a time you disagreed with a
      technical decision"
domain:"process"
category:"behavioral"
tags:
  -conflict_resolution
  -communication
  -technical_judgment
  -frequently_asked
  -core
  -коммуникация
difficulty:"middle"
roles:
  -"all_roles"
interview_stage:
  -"behavioral"
frequency:"high"
related:
  -"PROC-FRAMEWORK-001"    # STAR Method
  -"PROC-SOFTSKILL-002"    # Constructive Disagreement
  -"PROC-SOFTSKILL-005"    # Influence without authority
  -"PROC-PITFALL-007"      # Ошибка: критика без альтернативы
  -"PROC-COMPANY-001"      # Amazon (Leadership Principles:
                           #   Have Backbone; Disagree
                           #   and Commit)

prerequisites:
  -"PROC-FRAMEWORK-001"    # STAR Method
sources:
  -"Amazon Leadership Principles:
    https://www.amazon.jobs/en/principles"
  -"McDowell G.L., Bavaro J. Cracking the PM Interview.
    Chapter on Behavioral Questions. 2014."
author:"Инженер знаний №3"
version:"1.0"
date_created:"2025-03-08"
date_updated:"2025-03-08"
status:"approved"
---

# Tell me about a time you disagreed with a technical decision

## Что оценивает
- Способность отстаивать техническую позицию с аргументами
- Навык конструктивного конфликта (disagree and commit)
- Уважение к коллегам и процессу принятия решений
- Способность отделять эго от технической дискуссии
- Готовность принять чужое решение, если команда
  решила иначе

## Почему задают

```

Технические команды регулярно сталкиваются с разногласиями по архитектуре, выбору технологий, приоритетам. Компаниям важно, чтобы инженер умел конструктивно спорить, приводить data-driven аргументы и при этом не разрушать рабочие отношения. Amazon выделяет это в отдельный Leadership Principle: «Have Backbone; Disagree and Commit».

Структура ответа (STAR)

Situation

Выберите ситуацию, где:

- разногласие было **техническим** (не личным);
- ставки были достаточно высокими (влияние на продукт, производительность, надёжность);
- вы были **не единственным** участником решения.

Пример вводных: «В проекте X мы выбирали между подходом A и подходом B для...»

Task

Сформулируйте:

- что конкретно вы считали неправильным;
- почему это имело значение (impact);
- какова была ваша роль в принятии решения.

Action

Акцент на **процессе**, а не на победе:

1. Как вы сформулировали свою позицию (data, benchmarks, аналогии)
2. Как вы выслушали контраргументы
3. Как пришли к решению (escalation? POC? голосование?)
4. Что вы сделали **после** решения (особенно если решили не в вашу пользу)

Result

- Чем закончилось (конкретные метрики, если возможно)
- Что вы узнали
- Как изменились ваши взаимоотношения с коллегой

Пример хорошего ответа

«В проекте миграции базы данных мой техлид предложил перейти на MongoDB для всех сервисов. Я считал, что для нашего биллинг-сервиса с ACID-транзакциями это создаст проблемы.

Я подготовил сравнительную таблицу: какие сервисы выигрывают от документной модели, а какие проигрывают. Провёл нагрузочное тестирование биллинг-сервиса на PostgreSQL и MongoDB. Результаты показали, что на MongoDB мы теряем консистентность в edge cases.

Я представил данные на техническом ревью. Техлид согласился с аргументами, и мы приняли гибридное

решение: MongoDB для user-facing сервисов, PostgreSQL для биллинга. В итоге за 6 месяцев после миграции не было ни одного инцидента с консистентностью данных.

Я научился, что лучший аргумент в техническом споре – это данные, а не мнение.»

Что хорошо: конкретная ситуация, data-driven подход, конструктивный процесс, измеримый результат, рефлексия.

Пример плохого ответа

«Мой коллега хотел использовать React, а я – Vue.js. Я знал, что Vue лучше, потому что я на нём больше работал. В итоге мы использовали React, и проект получился не очень. Я до сих пор считаю, что нужно было выбрать Vue.»

Что плохо: нет данных, субъективные аргументы («я больше работал»), нет конструктивного процесса, нет рефлексии, обида после решения, нет «disagree and commit».

Вариации вопроса

- «Tell me about a time you had a conflict with a teammate about a technical approach»
- «Describe a situation where you pushed back on a decision»
- «When did you have to convince someone of your technical idea?»
- «Tell me about a time you were wrong about a technical decision» (инвертированная версия!)

Red Flags в ответах кандидатов

- 🚩 Переход на личности («он просто не понимал»)
- 🚩 Отсутствие data-driven аргументов
- 🚩 Неспособность принять чужое решение («и я оказался прав, а они нет»)
- 🚩 Escalation без попытки обсуждения
- 🚩 Избегание конфликтов («я всегда соглашаюсь с командой»)
- 🚩 Отсутствие рефлексии и выводов

Компании, которые часто задают

- **Amazon**: прямо соответствует Leadership Principles «Have Backbone; Disagree and Commit» + «Earn Trust»
- **Google**: «Googleness and Leadership» – конструктивное несогласие
- **Meta**: «Be Bold» + «Be Open» – баланс настойчивости и открытости
- **Любая компания уровня Senior+**: ожидается опыт технического лидерства

Связанные темы

- [[PROC-FRAMEWORK-001]] – STAR Method (depends_on)
- [[PROC-SOFTSKILL-002]] – Constructive Disagreement (relates_to)
- [[PROC-SOFTSKILL-005]] – Influence without authority (relates_to)
- [[PROC-PITFALL-007]] – Ошибка: критика без альтернативы (warns_about)
- [[PROC-COMPANY-001]] – Amazon Leadership Principles (relates_to)
- [[GLB-CROSS-003]] – Связь архитектурных решений и коммуникации (relates_to)

Источники

1. Amazon Leadership Principles.
<https://www.amazon.jobs/en/principles>
2. McDowell G.L., Bavaro J. Cracking the PM Interview. CareerCup, 2014.
3. Educative: Grokking the Behavioral Interview.
<https://www.educative.io/>